

LAB 3: Implementing inheritance - Documentation

The aim of this lab is to implement the design of Seminar 3, which focuses on building an application for linear regression using object-oriented programming concepts. We extended Lab 2 by adding new classes: Model, Algorithm, and SupervisedLearner and modifying the existing Vector class to include an augment() method. We also created a TestLearner class to verify that all the new components interact correctly.

The program as a whole should be able to represent a dataset, run a gradient descent algorithm to learn a linear regression model, and then predict outputs for new inputs.

We had to update:

1. **Vector:** Added the method `augment()` that returns a new vector with an additional 1 at the end to represent the bias term for linear regression.

We had to define these new classes:

2. **Model:** Represents the parameter vector θ of a linear model.
 - **Attributes:**
 - Vector params $\rightarrow$ parameter vector of dimension $d + 1$, where d is the input dimension.
 - **Constructor:**
 - Initializes θ to zero.
 - **Methods:**
 - `predict(Vector x)` $\rightarrow$ returns the scalar prediction $y^{\wedge} = \theta \cdot x$ (x must be augmented).
 - `update(Vector gradient, double alpha)` $\rightarrow$ updates the model parameters according to gradient descent $\theta = \theta - \alpha \nabla L(\theta)$.
 - `getParams()` $\rightarrow$ returns the parameter vector.
 - `toString()` $\rightarrow$ converts the model parameters to string for printing.
3. **Algorithm:** Implements gradient descent for linear regression.
 - **Attributes:**
 - double alpha $\rightarrow$ learning rate (controls the step size).
 - double stoppingCriterion $\rightarrow$ small threshold that determines when to stop iterating.
 - **Methods:**
 - `gradient(Dataset d, Model m)` $\rightarrow$ computes the gradient vector according to the formula from Seminar 3.
 - `solve(Dataset d)` $\rightarrow$ performs gradient descent by repeatedly computing the gradient and updating the model until convergence, and returns the learned Model.
4. **SupervisedLearner:** Manages all components of the learning process: the Dataset, the Algorithm, and the Model.
 - **Attributes:**
 - Algorithm algorithm (aggregation)
 - Dataset dataset (aggregation)
 - Model model (composition, created inside this class)
 - **Methods:**
 - `solve()` $\rightarrow$ delegates to the algorithm's `solve()` and stores the resulting model.

- `predict(Vector v)` → augments the non-augmented input vector and returns the prediction using the current model.
- `toString()` → returns a readable string of the model parameters (or a message if untrained).

5. **TestLearner:**

- Tests all the components by building a dataset, creating a learner, calling `solve()`, and verifying that predictions and model parameters behave as expected.
- Follows the same “works / does not work” style as previous labs, printing each test result and counting errors.

During the design phase, we discussed several possible implementations for the new classes and their relationships. One alternative was to implement gradient descent directly inside `SupervisedLearner`. This would have made the learner more self-contained but less flexible, since it would no longer be possible to reuse different optimization algorithms with the same learner. We therefore decided to let `SupervisedLearner` delegate to `Algorithm.solve()`, following a delegation pattern and emphasizing composition over inheritance.

We also discussed whether to pass an augmented dataset to the algorithm instead of augmenting each input on demand. We decided to augment inside the `predict()` and `gradient()` methods to keep the dataset independent of the regression implementation and reusable for other learners.

The theoretical object-oriented programming concepts we used for this lab:

- **Encapsulation** is used throughout the lab, all attributes are private and accessed only through getters or controlled updates.
- **Composition and aggregation** relationships are respected: `SupervisedLearner` *owns* its `Model` (composition), but only *aggregates* the `Algorithm` and `Dataset`, which can exist independently.
- **Delegation** and **abstraction** are used to make each class handle only its part of the process.
- **Polymorphism** was not required in the lab, but inheritance appears when reusing `Dataset` and its specialized versions (`StandardizedDataset`, `NormalizedDataset`) from Lab 2.

The final implementation worked well in practice. The `TestLearner` class confirmed that all methods behaved correctly:

- Before training, `toString()` displayed the “Untrained model (call `solve()` first)” message, as required.
- After calling `solve()`, the model parameters were successfully updated by gradient descent, and `toString()` printed the learned values.
- Predictions using `predict()` worked correctly for non-augmented input vectors, since the method internally performed the augmentation step.
- The results were consistent with expectations: for a dataset of zero outputs, the learned model remained approximately zero and predicted values close to 0.0, showing that the pipeline worked.

We encountered a few difficulties and doubts while implementing this lab.

The most confusing part at first was understanding the relations between the classes, especially why `SupervisedLearner` uses composition with `Model` but aggregation with `Dataset` and `Algorithm`. After reading the lab instructions and Seminar 3 notes again, we realized that the learner must create and own its `Model`, while the dataset and algorithm can exist on their own.

Another challenge was dealing with floating-point precision when checking predictions. Because of small numerical errors, values that should have been 0 sometimes appeared as very tiny numbers like $1e-12$. To avoid incorrect test failures, we added a small tolerance (epsilon) when comparing numbers, so two values count as equal if the difference is very small.

We also initially wondered what learning rate (α) and tolerance values to choose. After testing different combinations, we found that a learning rate of 0.01 and a tolerance of 0.000001 gave stable convergence without oscillations.

Additionally, we had some issues implementing the optional Moore-Penrose installing the EJML library despite attempting both the JAR-based setup and Gradle integration.

Finally, implementing the `SupervisedLearner.predict()` method required paying attention to augmenting the vector manually. Forgetting this step caused wrong predictions initially, until we added `Vector.augment()` inside the method.

Overall, the solution passed all tests with 0 errors. The classes integrated seamlessly with those from Lab 1 and Lab 2, showing a clear progression: from basic vector operations (Lab 1), to data handling and transformations (Lab 2), and finally to a working supervised learning framework (Lab 3). This lab helped us understand how composition, delegation and abstraction work in practice and showed us how theoretical design concepts translate into an actual learning algorithm in Java.